

Matching Compressor

Methods Description

1. Introduction

This document describes the methods used in **Matching Compressor**.

Before discussing the process of matching dynamic characteristics, it is necessary to describe the fundamentals of compression. This is covered in the next (second) section of the document. Since there is a lot of information available online about this topic, we will not go into detail, and those who are already familiar with compression methods can skip much of this section. At the end, we will introduce the concept of extended dynamic processing, specifically a multi-knee compressor with upward-expansion support. Considering multiple knees is important for more accurate matching of dynamic characteristics.

The third section is dedicated to methods for determining optimal processing parameters for matching dynamic characteristics. It begins by discussing the simplest algorithm for a compressor that does not use an envelope. After that, it covers an algorithm that takes attack and release into account and then presents a modification for multi-knee compression. Next, it discusses parameter selection and the quality measure. Finally, it presents an application of the method that keeps the level distribution unchanged when attack and release are retuned.

The source code of this plugin can be found at:

<https://github.com/justonem0reuser/MatchingCompressor>.

2. The Compression Model

All parameters in this section, unless otherwise specified, are measured in dB, except for the dimensionless R . Levels are likewise expressed in dB. A symbol that denotes an amplitude instead carries the subscript (amp); this occurs in the envelope filtering of Section 2.2.

2.1. Simplest Case

A compressor is designed to reduce the dynamic range of an audio signal. The simplest compressor can be described with the following formula:

$$L(x) = \begin{cases} G + x, & x \leq T \\ G + T + \frac{x-T}{R}, & x > T, \end{cases} \quad (1)$$

where:

L is the level of the processed signal (the output level),

x is the level of the input signal,

G is the gain,

T is the compression threshold,
 R is the compression ratio.

Thus, $L(x)$ is a piecewise linear function made up of two segments.

R can also be less than 1. In this case, the processor produces upward expansion. Note that classic (downward) expanders, conversely, process only samples with a level below the threshold, leaving others unchanged (only adding gain). For convenience, we will refer to dynamic signal processing as compression, regardless of the value of R .

To smooth the function $L(x)$, many compressors use Hermite quadratic interpolation (the so-called soft knee): in the vicinity of the break point, the original function is replaced with a quadratic function that touches both segments at points $T - W/2$ and $T + W/2$, where W is the knee width. In this case, the compression function $L(x)$ is:

$$L(x) = \begin{cases} G + x, & x \leq T - \frac{W}{2} \\ G + x + \left(\frac{1}{R} - 1\right) \frac{\left(x - T + \frac{W}{2}\right)^2}{2W}, & T - \frac{W}{2} < x < T + \frac{W}{2} \\ G + T + \frac{x - T}{R}, & x \geq T + \frac{W}{2} \end{cases} \quad (2)$$

This function has no breaks.

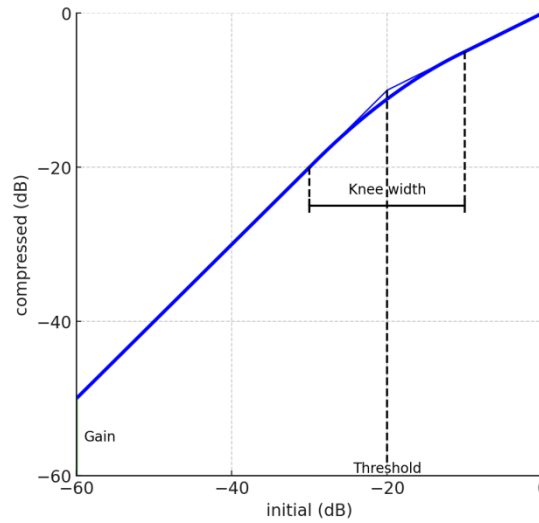


Fig. 1. Soft-knee compressor

2.2. Envelope

The compression function discussed above depends on one argument: with given compression parameters G , T , R , W , the level of the output sample depends only on the level of the input sample at the same moment. However, in practice, this type of compression does not yield satisfactory results: overload effects and other artifacts can occur, making the sound muddy.

To restore natural sound, the envelope is used to calculate the amount of compression. The envelope is obtained by pre-filtering that adds some "inertia" to the signal, smoothing out rapid changes in level. For the feedforward compressor described here, the output level is:

$$L(x, x_e) = L(x_e) + x - x_e. \quad (3)$$

The same symbol L is used for both forms: $L(x)$ is the single-argument compression function defined above, and $L(x, x_e)$ is its two-argument extension. Which form is meant follows from the number of arguments.

The envelope is obtained from the input signal, usually by EMA filtering. A classic EMA filter can be represented by the following formula:

$$x_{e\ n\ (amp)} = \alpha x_{n\ (amp)} + (1 - \alpha)x_{e\ n-1\ (amp)}, \quad 0 < \alpha \leq 1. \quad (4)$$

The application of filtering in compression has the following features:

1. This filter operates with amplitudes instead of levels in dB. The conversion formula is:

$$x_{(dB)} = 20 \log_{10} |x_{(amp)}| \quad (5)$$

2. The envelope represents the smoothed level – therefore, it is always non-negative regardless of the sign of the input sample. Depending on the type of envelope – peak or RMS – it is represented by one of the following formulas:

- for peak: $x_{e\ n\ (amp)} = \alpha |x_{n\ (amp)}| + (1 - \alpha)x_{e\ n-1\ (amp)}$;
- for RMS: $x_{e\ n\ (amp)} = \sqrt{\alpha x_{n\ (amp)}^2 + (1 - \alpha)x_{e\ n-1\ (amp)}^2}$.

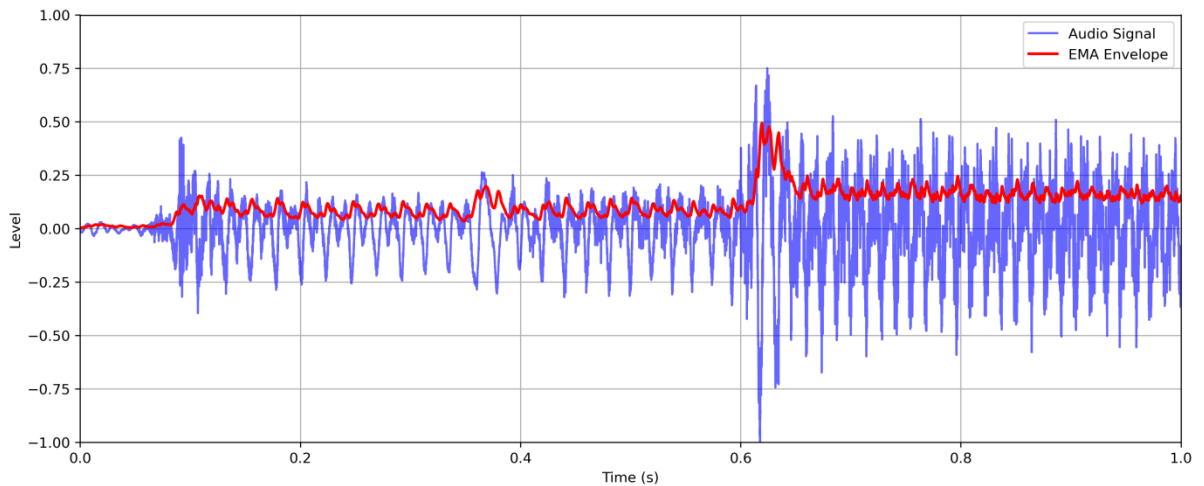


Fig. 2. Audio signal and its envelope

3. Often, a switchable smoothing coefficient is used during filtering:

$$\alpha = \begin{cases} \alpha_a(a), & x_{n\ (amp)} \geq x_{e\ n-1\ (amp)} \\ \alpha_r(r), & x_{n\ (amp)} < x_{e\ n-1\ (amp)} \end{cases} \quad (6)$$

The coefficients α_a , α_r are calculated based on the parameters a (attack) and r (release), which are usually specified in milliseconds.

Typically, the attack a is described as the time it takes for the compressor to fully engage (apply suppression) after the input signal level exceeds the threshold T , while the release r is the time it takes for the compressor to stop compressing the signal and return to the normal level after the signal level drops below the threshold. However, the term "fully" in these definitions is an oversimplification, as after applying the EMA filter, the compressor approaches the target level only asymptotically. A more precise definition depends on the nature of the dependencies $\alpha_a(a)$, $\alpha_r(r)$. In this implementation, the following formulas are used:

$$\alpha_a = e^{-\frac{2\pi}{af_s}}, \quad (7)$$

$$\alpha_r = e^{-\frac{2\pi}{rf_s}}, \quad (8)$$

where f_s is the sampling frequency of the audio data (samples per second). In this case, the attack time is the duration to reach about 99.8% of the target level (the release is defined similarly).

Thus, with unchanged compression parameters G , T , R , W , the average level of the processed signal increases with a longer attack time and decreases with a longer release time.

2.3. Processing Multi-Channel Audio

When processing multi-channel signals, there are three possible approaches:

- each channel is processed separately (using its own mono-compressor with the same parameters);
- the amount of compression is determined by the maximum smoothed level among all channels;
- the amount of compression is determined by the average smoothed level.

2.4. Multi-Knee Compressor

To more accurately match two audio signals, it is desirable to have a more complex dynamic curve. This can be achieved by considering a multi-knee compressor. The parameters of such a compressor include overall gain, attack, release, the type of multi-channel signal processing (separate, maximum or average), and the type of envelope (peak or RMS). Additionally, each knee has its own threshold, compression ratio, and knee width. This structure allows for more precise control over the dynamic characteristics of the audio signals being processed.

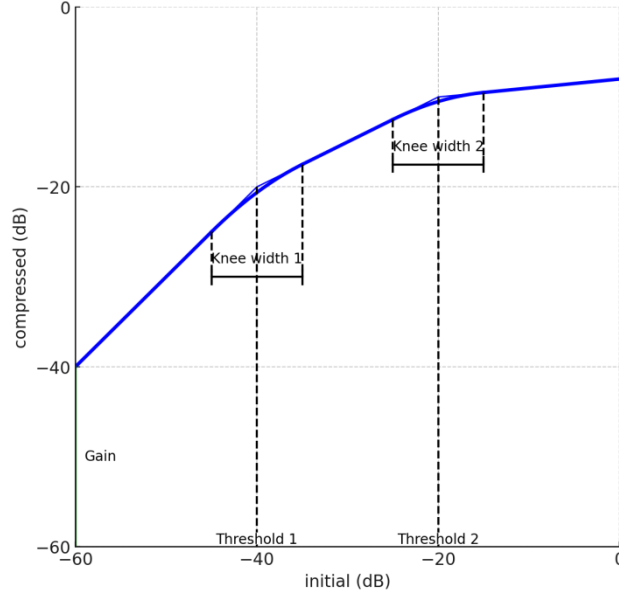


Fig. 3. Two-knee compressor

Let the knee index i be the highest index such that $x_e > T_i - 0.5W_i$;

$R_0 = 1$;

$T_{j-1} < T_j$ for each $j > 1$.

If $i = 0$, then no compression is applied, and

$$L(x_e) = x_e + G. \quad (9)$$

Otherwise

$$G_i = \begin{cases} G, & i = 1 \\ G + \sum_{j=2}^i \left(\frac{1}{R_{j-1}} - 1 \right) (T_j - T_{j-1}), & i > 1, \end{cases} \quad (10)$$

$$L(x_e) = \begin{cases} G_i + T_i + \frac{1}{R_{i-1}} (x_e - T_i) + \left(\frac{1}{R_i} - \frac{1}{R_{i-1}} \right) \frac{(x_e - T_i + \frac{W_i}{2})^2}{2W_i}, & x_e < T_i + \frac{W_i}{2} \\ G_i + T_i + \frac{1}{R_i} (x_e - T_i), & x_e \geq T_i + \frac{W_i}{2}. \end{cases} \quad (11)$$

3. Matching Dynamic Characteristics

Having reviewed the formulas used for signal compression, we now turn to the main task of this document: calculating the compression parameters that provide the best matching of dynamic characteristics between two audio fragments.

Unless otherwise specified, all levels in this section are dimensionless amplitudes rather than values in dB, while the parameters G , T and W remain expressed in dB. The compression function of Section 2 is therefore evaluated on an amplitude in three steps: the amplitude is

converted to dB with the conversion formula of Section 2.2, the function is applied, and the result is converted back to an amplitude.

3.1. The Simplest Case

First, let's consider the following scenario:

- the compressor consists of a single knee;
- the attack and release times are set to zero;
- each channel (in the case of multi-channel audio) is processed separately.

In this case, at any moment, the output of the compressor, given its known parameters, depends only on its input at that same moment: $L = L(x)$.

For each of the two audio fragments – the input and the reference – we can define a probability distribution function $F(x)$, where x represents the level. For each level x , the value $F(x)$ reflects the proportion of samples with a level not exceeding x . In other words, it is the probability that a random sample from the given fragment will have a level no greater than x . The function $F(x)$ is non-decreasing, with values ranging from 0 to 1. The task is to find the compressor parameters such that the resulting function of the processed audio, F_{comp} , closely matches the reference function F_{ref} .

The first question to address is: in what units should the signal level be measured?

Comparing values in decibels is not appropriate, as illustrated by the following example: suppose we need to compare two pairs of samples presented in the table below.

	I	II
x_{ref}	−100 dB	−1 dB
x_{comp}	−200 dB	−13 dB
Δx	100 dB	12 dB

From the table, we can conclude that the difference between −100 and −200 dB is much more significant. In reality, however, it is negligible because the levels themselves are extremely low. On the other hand, the difference between −1 and −13 dB (i.e., a factor of 4) is quite significant, as the levels are relatively high.

Therefore, levels should be compared by amplitudes – meaning we should use the units in which the information is stored in the audio file. In this case, the table would look like this:

	I	II
x_{ref}	10^{-5}	0.89
x_{comp}	10^{-10}	0.22
Δx	$\sim 10^{-5}$	0.67

Additionally, using decibels would require dealing with an infinite range of values from $-\infty$ to 0, while amplitude can take values from 0 to 1.

The second question is: how do we determine the degree of similarity between two distribution functions?

Experiments show that it is better to compare not the values of $F(x)$ for the average levels x of different intervals, but rather the values of quantiles x_q . These are the values at which the proportion of samples in the audio fragment that do not exceed x_q equals q : $F(x_q) = q$. In other words, this approach considers not the function F itself, but its inverse, F^{-1} .

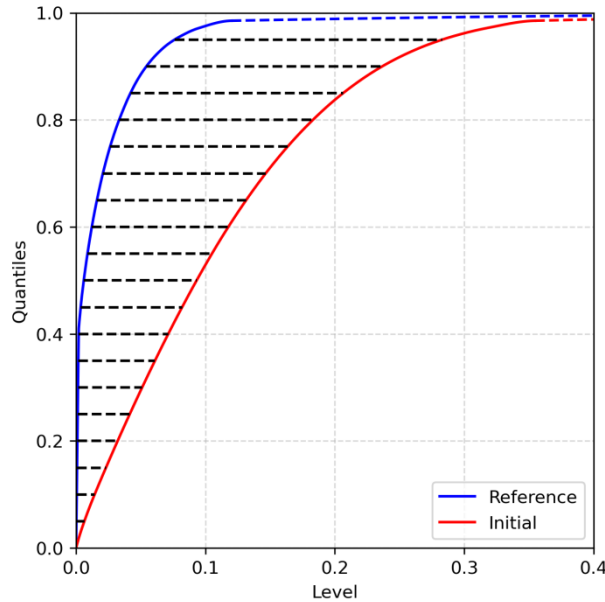


Fig. 4. Distribution matching

Let the number of points needed to discretize the function be N_q . Then the values q_j are calculated as:

$$q_j = \frac{j}{N_q + 1}, 1 \leq j \leq N_q. \quad (12)$$

For example, if we need to discretize the function into four points, these will be the levels below which 20%, 40%, 60%, and 80% of the samples lie.

To determine the quantiles for a given audio fragment, we need to:

- combine all audio channels into one, creating an array $\mathbf{a}$ of $N_s \cdot N_c$ values (N_s is the number of samples, N_c is the number of channels), storing not the actual values but their magnitudes (the level of a sample does not depend on its sign);
- sort the resulting array in ascending order;
- determine the indices of the desired elements in the array as follows:

$$k_j = \frac{N_s N_c j}{N_q + 1}, 1 \leq j \leq N_q. \quad (13)$$

If k_j is an integer, the desired value will be the k_j -th element of the array; otherwise, linear interpolation can be applied between the two neighboring elements.

The algorithm above defines the quantiles exactly. Its time complexity is determined by the time complexity of the sorting operation, which is $O(N \log N)$, where $N = N_s \cdot N_c$. It is possible to speed up the calculations by replacing the exact F^{-1} computation (if we disregard the error from linear interpolation between neighboring elements) with an approximate one. More importantly, the histogram-based form given below is needed in any case for the envelope, which is treated in Section 3.2, and the smooth way it spreads each sample over neighboring bins is what makes the gradient available analytically. Therefore F^{-1} is computed approximately, as follows:

- combine all audio channels into one array $\mathbf{a}$, just like in the previous algorithm;
- divide the amplitude range from 0 to 1 into N_G equal bins (discretization by level); let $\mathbf{f}$ be an array of size N_G holding the histogram, so that the sum of its elements is N ; each sample of $\mathbf{a}$ could be assigned to a single bin (increasing one element of $\mathbf{f}$ by 1), but for continuity and smoothness it is better to spread the sample over three neighboring bins using a quadratic B-spline kernel, with weights that sum to 1:
 - let k be the index of the bin nearest to the current sample of $\mathbf{a}$ and δ be the shift from the center of this bin to this sample relative to the bin width: $-0.5 \leq \delta \leq 0.5$;
 - then the three indices of elements to deposit a value are

$$\max(k - 1, 1); \quad k; \quad \min(k + 1, N_G)$$
 and their weights vector is

$$\mathbf{w} = \{0.5 \cdot (0.5 - \delta)^2; \quad 0.75 - \delta^2; \quad 0.5 \cdot (0.5 + \delta)^2\};$$
 - each of these three bins is increased by its corresponding weight;
- knowing the probability density function, calculate the probability distribution function multiplied by N : $F_1 = f_1$, $F_i = F_{i-1} + f_i$ (so that $F_{N_G} = N$);
- the quantile of order q_j corresponds to the target mass $Q_j = N \cdot q_j = N \cdot j / (N_q + 1)$; find the bin i for which $F_{i-1} < Q_j \leq F_i$ (with $F_0 = 0$) and obtain the quantile value by linear interpolation inside this bin:

$$x_{qj} = \frac{i-1}{N_G} + \frac{1}{N_G} \cdot \frac{(Q_j - F_{i-1})}{f_i}; \quad (14)$$

- as a safeguard against accumulated rounding, any trailing quantiles whose target mass is not reached are assigned the last computed value.

Since the deposit kernel is symmetric, F_i estimates the number of samples not exceeding the upper boundary of bin i , that is the amplitude i/N_G : a sample lying exactly on that boundary contributes exactly one half to F_i . The formula above is therefore a linear interpolation of the inverse distribution function between two consecutive bin boundaries.

The time complexity of this algorithm is $O(N)$. At the same time, it is possible to choose a sufficiently large N_G such that the accuracy does not significantly decrease.

By applying the described algorithm to the processed and reference audio fragments, we obtain two non-decreasing data arrays $\mathbf{x}_q$ and $\mathbf{y}_q$ of length N_q with values ranging from 0 to 1.

These two arrays serve as the input data for the task of determining the compression parameters.

Next, we need to solve the problem of finding the minimum sum of squared differences between the values of the reference array and the input array after being processed by the compressor with given parameters:

$$\sum_{j=1}^{N_q} \left(F_{\text{comp}}^{-1}(j) - F_{\text{ref}}^{-1}(j) \right)^2 \rightarrow \min \quad (15)$$

where F^{-1} is the inverse of the probability distribution function, with the subscript "ref" indicating reference values and the subscript "comp" indicating values after the compression of the input audio.

But $F_{\text{ref}}^{-1}(j) = y_{qj}$. Also, since in the problem at hand the output level at given compression parameters depends only on the input level at the same moment in time for the same channel, we have $F_{\text{comp}}^{-1}(j) \approx L(\mathbf{c}, x_{qj})$. Taking this into account, the problem can be reformulated as:

$$\sum_{j=1}^{N_q} \left(L(\mathbf{c}, x_{qj}) - y_{qj} \right)^2 \rightarrow \min. \quad (16)$$

The compressor parameters $\mathbf{c} = \{G, T, 1/R, W\}$ that achieve the minimum are the solution. Note that the unknown is the reciprocal $1/R$ rather than the ratio R itself, since the compression function depends on $1/R$ linearly. The elements of the array $\mathbf{c}$ are constrained by the minimum and maximum values of the corresponding parameter:

$$c_{j \min} \leq c_j \leq c_{j \max}. \quad (17)$$

Thus, the task is to approximate a known set of points (x_{qj}, y_{qj}) with a function of a given type. This is a standard nonlinear regression problem that can be solved using a variety of methods. As it is also a least squares problem, the Levenberg-Marquardt method with box constraints (17) and linear constraints is one of the most "natural" choices (the necessity of the linear constraints will be discussed in Section 3.3).

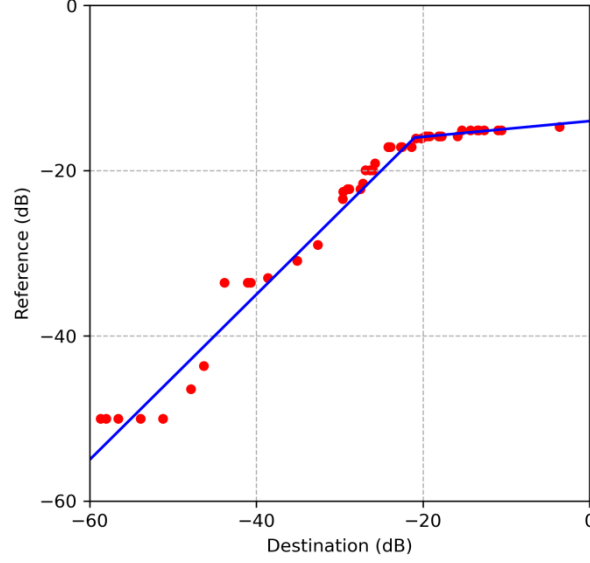


Fig. 5. Regression problem for single-knee mono compressor without envelope

It should be noted that the relationship between output level and input level at given parameter values is given in closed form. Therefore, for each set of parameters considered, it is not necessary to fully process the entire audio fragment, which significantly speeds up the solution. Instead, at each step of the algorithm, the output levels for N_q points are calculated using the known compression formulas.

Additionally, at each step of the algorithm, it is necessary to compute the gradient – the vector of partial derivatives of the compression function L with respect to each of the parameters:

$$\nabla L = \frac{\partial L}{\partial \mathbf{c}} = \left\{ \frac{\partial L}{\partial G}, \frac{\partial L}{\partial T}, \frac{\partial L}{\partial R}, \frac{\partial L}{\partial W} \right\}. \quad (18)$$

Since the compression formula $L(\mathbf{c}, x_{qj})$ is known, the gradient can be easily computed analytically. Experiments show that for audio fragments lasting about a minute, the described algorithm takes less than a second.

3.2. Matching with Consideration of the Envelope

Since the result of compression without using an envelope will be unsatisfactory, in real audio processing tasks, the attack and release times will be non-zero. If we calculate the compressor parameters that best match the processed audio with the reference audio based on their zero values and then increase them, the resulting level distribution will deviate from the reference. To avoid this, we need to reformulate the task: to determine the compressor parameters G, T, R, W such that after processing, the level distribution in the processed audio is as close as possible to the reference, given the specified values for attack, release, envelope type, and multi-channel audio processing type.

For the reference audio, the calculation of the inverse probability distribution function is done in the same way as described above. However, for the processed audio, the resulting level

function is now a function of two arguments: the original level and the envelope value. In this case, it is no longer possible to take N_q original levels of the processed signal and compute the output values. A similar situation arises when processing multi-channel audio, even with zero attack and release, if the amount of compression is determined not for each channel separately but based on the maximum or average level among all channels.

In this case, the task of finding the optimal compressor parameters is formulated as follows:

$$\sum_{j=1}^{N_q} (F_{\text{comp}}^{-1}(j, \mathbf{c}, \mathbf{x}, \mathbf{x}_e) - y_{qj})^2 \rightarrow \min \quad (19)$$

The entire audio fragment must be processed at each step to compute F_{comp}^{-1} . This, as well as the complexity of calculating the gradient, significantly reduces the speed of the algorithm.

To speed up the process while allowing for a permissible decrease in the accuracy of computing F_{comp}^{-1} , we can collect statistics on the input audio into a matrix $\mathbf{M}$ of size $N_G \times N_G$ before starting the iterations. Each value m_{ij} in this matrix should reflect the number of samples whose levels fall within the range $(i-1)/N_G \leq x < i/N_G$, while the corresponding envelope level at the same moment in time falls within the range $(j-1)/N_G \leq x_e < j/N_G$.

Then, at each step for computing F_{comp}^{-1} , we process this matrix instead of the audio fragment itself. An array $\mathbf{a}_{\text{dest}}$ is created in the same way as when processing the reference audio. For each non-zero m_{ij} , the value after compression $L(\mathbf{c}, (i-0.5)/N_G, (j-0.5)/N_G)$ is calculated. Based on this value, the indices of three elements in the array $\mathbf{a}_{\text{dest}}$ that need to be increased are determined, along with their quadratic B-spline weights. These three elements are increased by the value m_{ij} multiplied by the bin weight w . After filling $\mathbf{a}_{\text{dest}}$, the computation of F_{comp}^{-1} is performed in the same way as for the reference audio.

Besides $\mathbf{a}_{\text{dest}}$, a gradient can also be computed in a similar way. Each scalar $\mathbf{a}_{\text{dest}}$ bin corresponds to a gradient vector bin.

For each non-zero m_{ij} , the corresponding three gradient bins are increased not by the weights themselves but by their derivatives with respect to the compressed amplitude, multiplied by m_{ij} and by the derivative of that amplitude with respect to the parameter:

$$\frac{\partial f_k}{\partial c_p} += m_{ij} \frac{dw_k}{dL} \frac{\partial L}{\partial c_p}, \quad (20)$$

where

$$\frac{dw}{dL} = \{(\delta - 0.5) \cdot N_G; -2\delta \cdot N_G; (\delta + 0.5) \cdot N_G\}. \quad (21)$$

The quantile values have to be differentiated as well. Accumulating the gradient bins in the same way as the density gives $\partial F_i / \partial c_p$. Since the target masses Q_j do not depend on $\mathbf{c}$ (the total mass is conserved), differentiating the interpolation formula for x_{qj} gives

$$\frac{\partial x_{qj}}{\partial c_p} = \frac{1}{N_G f_i^2} \left(-\frac{\partial F_{i-1}}{\partial c_p} f_i - (Q_j - F_{i-1}) \cdot \frac{\partial f_i}{\partial c_p} \right) \quad (22)$$

with the same bin i that was used to obtain x_{qj} .

For the trailing quantiles filled by the last computed value, the gradient is copied along with this value.

A gain scales every sample by the same factor and therefore scales every quantile by that factor as well. The make-up gain can thus be applied to the quantiles once they have been obtained, instead of being propagated through the histogram, and its derivative is taken directly.

Thus, each step of minimization requires $O(N_G^2)$ time, assuming that $N_q < N_G$. This is justified if $N_G^2 \ll N_s N_c$. At the same time, N_G should be sufficiently large to maintain acceptable accuracy.

To optimize the process, it is reasonable to cache already computed arrays F_{comp}^{-1} .

3.3. Matching in the Case of a Multi-Knee Compressor

In the case where the compressor consists of N_K knees, the vector of computed compression parameters is represented as:

$$\mathbf{c} = \left\{ G, T_1, \frac{1}{R_1}, W_1, \dots, T_{N_K}, \frac{1}{R_{N_K}}, W_{N_K} \right\}. \quad (23)$$

To ensure that the knees are correctly ordered and that adjacent soft knees do not overlap, additional linear constraints should be added, one for each pair of adjacent knees, that is $N_K - 1$ in total:

$$T_{i-1} + 0.5W_{i-1} \leq T_i - 0.5W_i \quad (24)$$

where the threshold and knee width are expressed in dB.

These additional constraints can be used to solve problems both with and without an envelope.

Considering several knees increases the calculation time significantly.

3.4. Parameter Selection

It should be noted that the regression problem discussed above is nonlinear. Therefore, it is impossible to guarantee convergence to a global minimum. As a result, the following factors become very important:

- the parameters specified by the user;
- the initial values for the compression parameters (initialization);
- the tuning parameters for the minimization algorithm.

Regarding the user-specified parameters, as the number of unknowns increases, both the risk of missing the global minimum and the computation time also increase. Therefore:

- solving the problem with a large number of knees does not always yield a more accurate result;
- in most cases, it is advisable to solve the problem with a hard knee (with zero knee width) and then manually set the desired width: this will result in only minor changes to the level distribution while speeding up the solution.

For initial values of the compression parameters, one can suggest parameters where compression does not alter the data: $G = 0$, $R_i = 1$, $W_i = 0.5W_{\max}$ (in the case of a hard knee, $W_{\max} = 0$), $T_i = T_{\min} \cdot (N_K + 1 - i) / (N_K + 1)$ ($i = 1 \dots N_K$). However, another choice is also possible, depending on the level distribution in the input and reference audio.

The parameters of the minimization algorithm (which should either be set programmatically or provided for user selection) depend on the mathematical method used and its implementation. The choice of these tuning parameters should rely on experimental data. Here is a sample list:

- number of bins for level discretization;
- number of values (quantiles) used to discretize the minimized function;
- scaling coefficients;
- maximum number of iterations;
- maximum step vector size at which the solution is considered found.

When choosing discretization values, one should take into account the following:

- The number of level bins must be chosen so that each bin is determined by at least dozens of samples on average; therefore, for short audio fragments this number should not be too large.
- The soft-knee case requires many more level bins, as it deals with subtle effects in the vicinity of the threshold. In this case the number of level bins should not depend on the number of knees.
- For the hard-knee case, the number of quantiles should be the primary consideration: the more knees are taken into account, the more quantiles are needed to ensure acceptable precision.
- When using quadratic B-spline interpolation, the number of occupied level bins should be at least three times the number of quantiles to ensure that the quantiles are independent.

3.5. Measuring the Quality of the Solution

In (19) the quantity minimized is a sum over all quantiles: at each quantile the difference between the processed and the reference level is squared. Both levels are dimensionless. For display a derived measure is more convenient – the square root of that sum, divided by the root sum of squares of the reference levels:

$$S = \sqrt{\frac{\sum_{j=1}^{N_q} (F_{\text{comp}}^{-1}(j, c, x, x_e) - y_{qj})^2}{\sum_{j=1}^{N_q} (y_{qj})^2}} \quad (25)$$

This makes the measure independent of the loudness of the reference. It is bounded below by zero and unbounded above, zero corresponding to a perfect fit.

The measure is not symmetric, however. For deviations toward lower levels it is bounded above by one, that value being reached when the processed audio is complete silence, whereas for deviations toward higher levels there is no upper bound.

3.6. Changing the Envelope Parameters While Preserving the Level Distribution

The method described above has a further application. When the envelope parameters are changed, it can be used to compute compression parameters that keep the resulting change in the level distribution to a minimum. To do this, instead of using another audio fragment as the reference, the fragment being processed itself should be used, compressed with the original compression and envelope parameters. Changing the attack and release values then yields the compression parameters that disturb the level distribution least.

Appendix. Notation

Bold symbols denote arrays and vectors. For levels the dimension follows the convention stated at the beginning of Sections 2 and 3: values are in dB in Section 2 and amplitudes in Section 3, unless a symbol carries the subscript (amp) or (dB).

Symbol	Meaning	Dimension or range
Compression function and its parameters		
L	output level produced by the compression function; $L(x)$ is the single-argument form, $L(x, x_e)$ the two-argument one	dB / amplitude
x	level of the input sample	dB / amplitude
x_e	envelope level	dB / amplitude
$x_{(dB)}, x_{(amp)}$	the same level expressed in dB and as an amplitude	—
G	gain	dB
T	compression threshold	dB
W	knee width	dB
R	compression ratio; $R < 1$ gives expansion	dimensionless
$\mathbf{c}$	vector of the compression parameters $\{G, T, 1/R, W\}$	—
$c_j, c_{j \min}, c_{j \max}$	the j -th parameter of $\mathbf{c}$ and its bounds	—
Multi-knee compressor (Section 2.4)		
N_K	number of knees	—
i	knee index; $i = 0$ means that no compression is applied	$0 \dots N_K$
T_i, W_i, R_i	threshold, knee width and ratio of the i -th knee	dB, dB, dimensionless
R_0	ratio below the first knee, equal to 1	dimensionless
G_i	gain accumulated up to the i -th knee	dB
Envelope (Section 2.2)		
n	sample index	—
α	EMA smoothing coefficient	$0 < \alpha \leq 1$
α_a, α_r	smoothing coefficients used for attack and for release	$0 < \alpha \leq 1$
a, r	attack and release times	ms
f_s	sampling frequency	samples per second
Audio data		
N_s	number of samples in a fragment	—
N_c	number of channels	—
N	total number of values, $N = N_s \cdot N_c$	—
$\mathbf{a}$	array holding all channels combined, storing sample magnitudes	amplitude
$\mathbf{a}_{\text{dest}}$	array of the same kind built from the compressed values	amplitude
Distributions and quantiles (Section 3.1)		
$F(x)$	probability distribution function of the levels	$0 \dots 1$
F^{-1}	inverse of the distribution function	—
$F_{\text{comp}}, F_{\text{ref}}$	distribution functions of the processed and of the reference fragment	—
N_q	number of quantiles	—
j	quantile index	$1 \dots N_q$
q_j	probability corresponding to the j -th quantile	$0 \dots 1$
x_{qj}, y_{qj}	j -th quantile of the processed and of the reference fragment	amplitude
k_j	index of the array element holding the j -th quantile	—
Histogram and statistics (Sections 3.1–3.2)		
N_G	number of level bins	—

Symbol	Meaning	Dimension or range
i	bin index	$1 \dots N_G$
f	histogram; f_i is the content of the i -th bin	—
F_i	cumulative histogram, $F_i = F_{i-1} + f_i$	$0 \dots N$
Q_j	target mass of the j -th quantile, $Q_j = N \cdot q_j$	—
k	index of the bin nearest to the current sample	—
δ	shift of the sample from the bin center, in bin widths	$-0.5 \dots 0.5$
$\mathbf{w}$	vector of the three quadratic B-spline weights; w_k is one of them	sum equals 1
M	matrix of the joint statistics of level and envelope	—
m_{ij}	element of M	—
	Gradient (Sections 3.1–3.2)	
∇L	gradient of the compression function with respect to $\mathbf{c}$	—
p	index of a parameter within $\mathbf{c}$	—
	Solution quality (Section 3.5)	
S	relative deviation of the processed levels from the reference; zero means a perfect fit	$0 \dots \infty$